

claude-windows / 985cc286-9e1e-4217-87c3-2d418bc4fa9c

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\985cc286-9e1e-4217-87c3-2d418bc4fa9c\subagents\workflows\wf_3e301d0b-e59\agent-a6954a70357e4059d.jsonl`  
- SHA-256: `b4f72cac6c1e18c67d1b6abe24729748c9246ecef96cdd6cdc5e288f2dd1fc6b`  
- Source modified: `2026-06-13T18:46:55+00:00`  
- Imported at: `2026-07-05T16:48:25+00:00`  
- project: `wf_3e301d0b-e59`  
- session_id: `985cc286-9e1e-4217-87c3-2d418bc4fa9c`
```

Transcript

1. user (2026-06-13T18:45:07.537Z)

Map the live shuttle segmenter path so we know EXACTLY where a person cue + Gate A?B plug in. Read: backend/pipeline/segmenters/trajectory_hybrid.py (TrajectoryHybridSegmenter), wasb_hybrid.py, tracknet_hybrid.py, and backend/pipeline/detectors/base.py (DetectorRunner + parse_trajectory_csv / trajectory_to_action_windows). Report: the 4-phase flow (healthcheck -> P2 trajectory->windows(+persist) -> P3 provider handoff -> P4 DB/link); the exact method that seeds windows from the trajectory; the candidate persistence call + the stats JSON schema written; the subclass contract (family, _build_runner, display_label); and the precise seam a FusionSegmenter should extend or compose. Repo root = C:/Users/avidu/Projects/badminton-highlight-indexer (the indexer). Read the ACTUAL files and quote file:line. Be precise, not generic.

2. assistant (2026-06-13T18:45:09.184Z)

I'll map the live shuttle segmenter path by reading the actual files you've specified. Let me start with a comprehensive exploration.

3. user (2026-06-13T18:45:09.436Z)

```
1      """Shared engine for trajectory-substrate hybrid segmenters.  
2  
3      tracknet_hybrid and wasb_hybrid were copy-paste twins ? CODE_MAP carried a  
4      standing footgun: "any P3/P4 fix must be applied to BOTH". This base class is  
5      now the single home for the whole pipeline shape:  
6  
7          healthcheck gate -> P2 trajectory -> candidate windows (+ persistence)  
8          -> P3 AI-provider handoff over padded clips -> P4 DB population/linking  
9  
10     A concrete subclass supplies only its identity and runner wiring:  
11     - ``family`` ? config section under ``indexing.*``, output subdir, and  
12       the ``<family>-hybrid-<model>`` detector tag in metadata.  
13     - ``display_label`` ? human label for log lines ("WASB", "TrackNet").  
14     - ``name`` / ``description`` properties (the registry contract).  
15     - ``_build_runner(idx_cfg)`` ? returns (DetectorRunner, detector-specific  
16       detection_params extras such as weights_path).  
17  
18     This base is deliberately NOT registered; only concrete subclasses register.  
19     """  
20     import os  
21     import time  
22     import logging  
23     import concurrent.futures  
24     from typing import Any, Dict, List, Optional, Tuple  
25  
26     import cv2
```

```

27
28 from backend.pipeline.segmenters.base import BasePlaySegmenter
29 from backend.utils.video import get_video_duration, extract_video_chunk
30 from backend.sports import sport_registry
31 from backend.providers import provider_registry
32 from backend.pipeline.detectors.base import DetectorRunner
33
34 logger = logging.getLogger(__name__)
35
36
37 def _fmt_ts(seconds: float) -> str:
38     seconds = max(0, int(seconds))
39     h, rem = divmod(seconds, 3600)
40     m, s = divmod(rem, 60)
41     return f"{h:02d}:{m:02d}:{s:02d}"
42
43
44 class TrajectoryHybridSegmenter(BasePlaySegmenter):
45     """Trajectory-detector hybrid: precise candidate rallies + AI semantic
boundaries."""
46
47     #: config section under `indexing` (velocity_threshold lives there), the
48     #: output subdir for trajectories, and the metadata detector-tag prefix.
49     family: str = ""
50     #: human-readable label used in log lines.
51     display_label: str = ""
52
53     def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
Any]]:
54         """Return (runner, detector-specific detection_params extras)."""
55         raise NotImplementedError
56
57     @staticmethod
58     def _probe_fps_width(video_path: str) -> tuple:
59         """Return (fps, frame_width) for a video, with sensible fallbacks."""
60         cap = cv2.VideoCapture(video_path)
61         fps = cap.get(cv2.CAP_PROP_FPS) if cap.isOpened() else 0.0
62         width = cap.get(cv2.CAP_PROP_FRAME_WIDTH) if cap.isOpened() else 0.0
63         cap.release()
64         return (fps if fps > 0 else 30.0, width if width > 0 else 1280.0)
65
66     @staticmethod
67     def _shot_count_from(segment: Dict[str, Any], duration: float) -> int:
68         """Provider-reported exchange count, else a duration-based estimate.
69
70         One canonical implementation ? the twins had drifted (wasb relied on
71         ``int(None)`` raising into the fallback; same result, by accident).
72         """
73         shot_count = segment.get("shuttle_exchange_count")
74         if shot_count is None:
75             return max(3, int(duration / 0.8))
76         try:
77             return int(shot_count)
78         except (ValueError, TypeError):
79             return max(3, int(duration / 0.8))
80
81     def process_video(self, video_id: str, video_path: str, # type: ignore[override]
82                     player_pool: Optional[List[str]] = None,
83                     max_frames: Optional[int] = None) -> List[Dict[str, Any]]:
84         # override-ignore: the ABC declares the Result contract (Success|Failure)
85         # but every live segmenter still returns a bare list ? the documented
86         # deliberate-incremental gap (CODE_MAP gotchas; main.py handles both).
87         label = self.display_label
88         # --- Sport profile + AI provider wiring (identical across segmenters) ---
89         sport_name = self.config.get("sport", "badminton")

```

```

90         try:
91             sport_profile = sport_registry.get(sport_name)
92         except KeyError as e:
93             logger.error(str(e))
94             return []
95
96         idx_cfg = self.config.get("indexing", {})
97         provider_name = self.config.get("ai_provider", idx_cfg.get("ai_provider",
"gemini"))
98         model_name = self.config.get("ai_model", idx_cfg.get("ai_model",
"gemini-2.5-flash"))
99
100         api_key_env_var = f"{provider_name.upper()}_API_KEY"
101         api_key = os.environ.get(api_key_env_var)
102         if not api_key:
103             logger.error(
104                 f"{api_key_env_var} environment variable is not set! "
105                 f"To run the {provider_name} handoff, set your API key. "
106                 f"In PowerShell: $env:{api_key_env_var}=\"your_api_key_here\""
107             )
108             return []
109         try:
110             provider = provider_registry.get(provider_name, api_key=api_key,
model_name=model_name)
111         except KeyError as e:
112             logger.error(str(e))
113             return []
114
115         video_duration = get_video_duration(video_path)
116         if video_duration <= 0:
117             logger.error("Invalid video duration.")
118             return []
119         fps, frame_width = self._probe_fps_width(video_path)
120
121         # --- Runner config + windowing thresholds ---
122         runner, runner_params = self._build_runner(idx_cfg)
123
124         velocity_thresh = idx_cfg.get(self.family, {}).get("velocity_threshold", 0.02)
125         merge_gap = idx_cfg.get("dead_time_merge_gap", 2.0)
126         min_window_duration = idx_cfg.get("min_action_window_duration", 2.0)
127         handoff_padding = idx_cfg.get("ai_handoff_padding", 2.0)
128         ai_max_workers = idx_cfg.get("ai_max_workers", 5)
129         log_candidates = idx_cfg.get("log_yolo_candidates", True)
130         store_candidates = idx_cfg.get("store_yolo_candidates", True)
131
132         detection_params = {
133             "detector": self.name,
134             "velocity_thresh": velocity_thresh,
135             "merge_gap": merge_gap,
136             "min_action_window_duration": min_window_duration,
137             **runner_params,
138         }
139
140         # --- Phase 1: env healthcheck (fail fast with a clear message) ---
141         ok, msg = runner.healthcheck()
142         if not ok:
143             logger.error("%s WSL environment not ready: %s", label, msg)
144             return []
145         logger.info("%s WSL env OK: %s", label, msg)
146
147         # --- Phase 2: trajectory -> candidate rally windows ---
148         # Trajectory detectors process the whole video in one pass (they carry
149         # their own frame buffering), so unlike yolo_hybrid we don't pre-chunk.
150         t_start = time.time()
151         out_dir = os.path.join("output", self.family)

```

```

152         csv_path = runner.run_predict(video_path, out_dir, video_id=video_id)
153         if not csv_path:
154             logger.error("%s produced no trajectory; aborting.", label)
155             return []
156
157         points = runner.parse_trajectory_csv(csv_path)
158         logger.info("Parsed %d trajectory points (%d visible).",
159                     len(points), sum(1 for p in points if p.visible))
160
161         all_candidate_windows = runner.trajectory_to_action_windows(
162             points, fps=fps, frame_width=frame_width, chunk_start=0.0,
163             velocity_thresh=velocity_thresh, merge_gap=merge_gap,
164             min_window_duration=min_window_duration, chunk_index=0,
165         )
166         logger.info("Phase 2 (%s) completed in %.1fs. Candidate windows: %d",
167                     label, time.time() - t_start, len(all_candidate_windows))
168
169         if log_candidates:
170             for cw in all_candidate_windows:
171                 logger.info(f"[{label} rally]
172 {_fmt_ts(cw['start'])}-{_fmt_ts(cw['end'])} "
173                             f"({cw['end'] - cw['start']:.1f}s) |
174 frames={cw['active_frame_count']} "
175                             f"peak_v={cw['peak_velocity']:.3f}
176 mean_v={cw['mean_velocity']:.3f}")
177
178         # Persist raw candidates for the fine-tuning dataset (same table as YOLO).
179         candidate_ids: List[Optional[int]] = [None] * len(all_candidate_windows)
180         if store_candidates:
181             for i, cw in enumerate(all_candidate_windows):
182                 stats = {
183                     "peak_velocity": cw["peak_velocity"],
184                     "mean_velocity": cw["mean_velocity"],
185                     "active_frame_count": cw["active_frame_count"],
186                     "track_id_count": cw["track_id_count"],
187                 }
188                 candidate_ids[i] = self.db.add_candidate_segment(
189                     video_id, detector=self.name,
190                     start_time=cw["start"], end_time=cw["end"],
191                     chunk_index=cw["chunk_index"], confidence=min(1.0,
192 cw["peak_velocity"]),
193                     stats=stats, detection_params=detection_params,
194                 )
195
196         if not all_candidate_windows:
197             return []
198
199         # --- Phase 3: AI provider handoff over padded candidate clips ---
200         handoff_dir = os.path.join("output", f"chunks_{provider_name}_handoff")
201         os.makedirs(handoff_dir, exist_ok=True)
202         logger.info("Phase 3: %s validation on %d candidate windows...",
203                     provider_name, len(all_candidate_windows))
204
205         def process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:
206             w_start = max(0, window["start"] - handoff_padding)
207             w_end = min(video_duration, window["end"] + handoff_padding)
208             w_dur = w_end - w_start
209             w_path = os.path.join(handoff_dir, f"handoff_{video_id}_{wi}.mp4")
210             if not extract_video_chunk(video_path, w_start, w_dur, w_path,
211 reencode=True):
212                 return []
213             prompt = sport_profile.get_prompt(chunk_duration=w_dur)
214             segments, _ = provider.analyze_video(w_path, prompt, chunk_duration=w_dur,
215                                                 label=f"Handoff {wi+1}")
216             valid = []

```

```

212         for seg in segments:
213             try:
214                 seg["start_time"] = float(seg["start_time"]) + w_start
215                 seg["end_time"] = float(seg["end_time"]) + w_start
216                 seg["_candidate_id"] = candidate_ids[wi]
217                 valid.append(seg)
218             except (ValueError, TypeError, KeyError) as e:
219                 logger.warning("Dropping segment with unparseable timestamps: %s ?
%s", seg, e)
220             try:
221                 os.remove(w_path)
222             except OSError:
223                 pass
224             return valid
225
226         ai_segments: List[dict] = []
227         with concurrent.futures.ThreadPoolExecutor(max_workers=ai_max_workers) as
executor:
228             futures = [executor.submit(process_candidate_window, wi, cw)
229                          for wi, cw in enumerate(all_candidate_windows)]
230             for future in futures:
231                 ai_segments.extend(future.result())
232             try:
233                 os.rmdir(handoff_dir)
234             except OSError:
235                 pass
236             logger.info("Phase 3 completed. AI confirmed %d semantic play segments.",
len(ai_segments))
237
238         # --- Phase 4: DB population + candidate linking ---
239         segments_data = []
240         for segment in ai_segments:
241             try:
242                 start = float(segment["start_time"])
243                 end = float(segment["end_time"])
244                 dur = end - start
245                 metadata = {
246                     "shot_count": self._shot_count_from(segment, dur),
247                     "shot_count_confidence": segment.get("shot_count_confidence",
"Unknown"),
248                     "detector": f"{self.family}-hybrid-{model_name}",
249                 }
250                 segment_id = self.db.add_play_segment(video_id, start, end, None, None,
metadata)
251                 if segment_id:
252                     candidate_id = segment.get("_candidate_id")
253                     if candidate_id is not None:
254                         self.db.link_candidate_to_segment(candidate_id, segment_id)
255                     segments_data.append({
256                         "id": segment_id, "start_time": start, "end_time": end,
257                         "duration": dur, "metadata": metadata,
258                     })
259             except Exception as e:
260                 logger.warning("Skipping invalid segment struct: %s", e)
261
262         return segments_data
263

```

4. user (2026-06-13T18:45:09.862Z)

```

1     """Abstract base for detector runners (TrackNet, WASB, future native, etc.).
2
3     This completes the Detector Runner Abstraction (low-regret item 6 continuation / item 1
4     of the post-observability re-prioritization) to de-risk the WSL/conda dependency for
5     the high-accuracy shuttle trajectory substrate used by tracknet_hybrid / wasb_hybrid.

```

```

6
7 The ABC lets us:
8 - Keep the current WSL adapters working unchanged for Windows users.
9 - Swap in (or A/B test) a Linux-native implementation later with zero changes to
10 the hybrid segmenters or the model-agnostic windowing brain.
11 - Provide a single place to evolve the healthcheck / error contract and thread
12 detector readiness into per-video reports.
13
14 Contract (deliberately matches what the concrete runners + hybrids + tests already did):
15 - run_predict(video_win_path: str, output_win_dir: str) -> Optional[str]
16   Run detector, return the Windows-side path to the produced trajectory CSV
17   (e.g. ".../clip_predict.csv" or ".../clip_wasb.csv"), or None on any failure.
18   The caller (hybrid) is responsible for output dir; impls may stage the video
19   into WSL fs for perf.
20
21 - healthcheck() -> tuple[bool, str]
22   Return (ok, message). Never raises. "ok" decides whether to proceed to run_predict.
23   Message is logged (and can be surfaced to reports or UX in future).
24   Intended to be cheap (env existence, import, GPU visibility, weights present).
25
26 Existing behavior is 100% preserved. The only additions are the inheritance and
27 the explicit common type for future implementers.
28
29 How to add a new runner (for docs / future contributors):
30 1. Subclass DetectorRunner in a new or existing module under pipeline/detectors/.
31 2. Implement run_predict and healthcheck (return (bool, str) tuple).
32 3. If it produces the same CSV schema (Frame,Visibility,X,Y), reuse or re-export
33   TrackNetRunner.parse_trajectory_csv and .trajectory_to_action_windows (they are
34   deliberately static and model-agnostic). If the new detector needs different
35   windowing, it can implement its own or we generalize later.
36 4. Lazy-import heavy native deps inside the methods (so importing the segmenter
37   registry never pulls torch/tf on a machine that doesn't have them).
38 5. Update the two hybrid segmenters (or introduce a detector= choice) to accept
39   DetectorRunner instances (they already do the right thing via duck-typing + the
40   type hints we add here).
41 6. Add unit tests that exercise the contract (isinstance, healthcheck shape, run
42   returns str|None) using mocks for the external bits ? see test_tracknet_runner.py.
43 7. No change to eval/calibration code paths (they consume the CSV + the static
44   windowing helpers directly).
45
46 Optional Linux-native path (the main de-risk goal):
47 A LinuxNativeRunner (or ONNXRunner) would:
48 - Skip all wsl / _stage / conda activate.
49 - Load weights locally (torch or onnx) or call a native binary.
50 - Write the exact same CSV format so trajectory_to_action_windows works verbatim.
51 - healthcheck can just check "weights file exists and torch importable + CUDA?".
52 This removes the WSL requirement for Linux users / future SaaS workers while
53 keeping the exact same candidate quality characteristics (or better, once native
54 weights are available).
55
56 Error surfacing into reports:
57 The healthcheck message is already logged by the hybrids on failure. The reporting
58 harvest sees the outcome as "0 candidates" + detector name. When we wire richer
59 segmenter Results (or pass detector_health into emit_indexer_report context), the
60 (ok, msg) can be recorded explicitly under segmentation.details or a new detector
61 stage. The ABC makes that future change local to one place.
62
63 See also:
64 - backend/pipeline/segmenters/{tracknet_hybrid,wasb_hybrid}.py (the two callers)
65 - tests/test_tracknet_runner.py (contract + pure logic tests; hybrids mock the runner)
66 - docs/TRACKNET_WSL_SETUP.md and CODE_MAP.md (2.2 Detectors section)
67 ""
68
69 import logging
70 import shlex

```

```

71 import subprocess
72 from abc import ABC, abstractmethod
73 from typing import Any, Callable, List, Optional, Tuple
74
75 logger = logging.getLogger(__name__)
76
77
78 def wsl_tilde_quote(path: str) -> str:
79     """shlex.quote that still lets bash expand a leading `~/`.
80
81     ``shlex.quote("~/clips/x.mp4")`` single-quotes the whole path, so bash
82     treats ``~`` literally and ``cp``/``mkdir`` fail with "No such file or
83     directory" ? this broke live WSL staging for the default ``~/clips``
84     stage dir (found by the 2026-06-11 wasb_hybrid smoke run; the BACKLOG
85     "shlex-quote breaks ~ expansion" item). Quoting only the part after
86     ``~/`` keeps expansion AND protects spaces/specials in the remainder.
87     """
88     if path == "~":
89         return path
90     if path.startswith("~/"):
91         return "~/ " + shlex.quote(path[2:])
92     return shlex.quote(path)
93
94
95 class WslCommandMixin:
96     """Shared ``wsl -d <distro> bash -c 'source <conda_sh> && <cmd>'`` invocation.
97
98     Lives beside (not on) DetectorRunner: a future Linux-native runner must not
99     inherit WSL plumbing. Requires ``self.cfg`` with ``distro``, ``conda_sh``
100     and ``timeout_sec`` attributes (TrackNetConfig and WasbConfig both qualify).
101     ``timeout_sec == 0`` maps to None = wait forever (configs default 1800s).
102     """
103
104     cfg: Any
105
106     def _wsl_bash(self, inner_cmd: str) -> subprocess.CompletedProcess:
107         full = f"source {self.cfg.conda_sh} && {inner_cmd}"
108         argv = ["wsl", "-d", self.cfg.distro, "bash", "-c", full]
109         logger.debug("WSL exec: %s", full)
110         return subprocess.run(argv, capture_output=True, text=True,
111                               timeout=(self.cfg.timeout_sec or None))
112
113     # ---- cache hygiene helpers (shared by both WSL runners) -----
114     def _wsl_free_gb(self, wsl_path: str) -> Optional[float]:
115         """Best-effort free space (GB) on the WSL filesystem holding ``wsl_path``.
116
117         Returns None if it can't be determined (never raises). Used as a pre-run
118         disk guard so a long extraction doesn't fill the disk mid-flight.
119         """
120         try:
121             res = self._wsl_bash(f"df -P -B1 {wsl_tilde_quote(wsl_path)} | awk
122 'NR==2{{print $4}}'")
123         except (subprocess.TimeoutExpired, OSError):
124             return None
125         try:
126             return int((res.stdout or "").strip()) / (1024 ** 3)
127         except (ValueError, AttributeError):
128             return None
129
130     def _wsl_rm_rf(self, wsl_paths: List[str]) -> None:
131         """Best-effort recursive delete of WSL paths (post-success cleanup; never
132         raises)."""
133         paths = [p for p in wsl_paths if p]
134         if not paths:
135             return

```

```

134         quoted = " ".join(wsl_tilde_quote(p) for p in paths)
135         try:
136             self._wsl_bash(f"rm -rf {quoted}")
137         except (subprocess.TimeoutExpired, OSError) as e:
138             logger.warning("WSL cache cleanup failed (non-fatal): %s", e)
139
140
141     class DetectorRunner(ABC):
142         """Common interface for shuttle/ball trajectory detectors (WSL or otherwise)."""
143
144         @abstractmethod
145         def run_predict(self, video_win_path: str, output_win_dir: str,
146                       video_id: Optional[str] = None) -> Optional[str]:
147             """Run inference and return path to trajectory CSV (or None on failure).
148
149             ``video_id`` (the file MD5) namespaces all staged/cached/output artifacts so two
150             videos that happen to share a filename cannot reuse each other's frames or CSV.
151             It is optional for back-compat; callers in the live pipeline always pass it.
152             """
153             pass
154
155         @abstractmethod
156         def healthcheck(self) -> Tuple[bool, str]:
157             """Quick check if the runner (env, weights, etc.) is usable.
158
159             Returns (ok, message). Never raises. Cheap to call before a long run.
160             """
161             pass
162
163         # De facto trajectory contract: the trajectory hybrids call these
164         # unconditionally after run_predict. Concrete runners reuse/re-export the
165         # model-agnostic TrackNetRunner staticmethods (same CSV schema + windowing);
166         # a runner with custom windowing supplies its own with the same shapes.
167         parse_trajectory_csv: Callable[..., Any]
168         trajectory_to_action_windows: Callable[..., Any]
169

```

5. user (2026-06-13T18:45:10.001Z)

```

backend\pipeline\segmenters\tracknet_hybrid.py
backend\pipeline\segmenters\wasb_hybrid.py
backend\pipeline\segmenters\trajectory_hybrid.py
backend\pipeline\segmenters\yolo_hybrid.py

```

6. user (2026-06-13T18:45:10.087Z)

```

backend\pipeline\detectors\rally_gate.py
backend\pipeline\detectors\__init__.py
backend\pipeline\detectors\wasb_infer.py
backend\pipeline\detectors\base.py
backend\pipeline\detectors\tracknet_runner.py
backend\pipeline\detectors\wasb_runner.py
backend\pipeline\detectors\person_detector.py

```

7. assistant (2026-06-13T18:45:11.537Z)

Now let me read the wasb_hybrid and tracknet_hybrid files to understand the concrete implementations:

8. user (2026-06-13T18:45:12.190Z)

```

1         """WASB-based hybrid play segmenter.
2
3         Phase 2 uses **WASB** (nttcom/WASB-SBDT, MIT) shuttle-trajectory detection via

```



```

4      WSL2 (backend/pipeline/detectors/wasb_runner.py). WASB ships pretrained
5      badminton weights and runs today, so this is the live shuttle-motion substrate.
6
7      For the per-video *calibration* goal we mainly use Phase 2 (trajectory ->
8      candidate windows); the windowing thresholds (velocity_thresh / merge_gap /
9      min_window_duration) are what get tuned against golden rally labels.
10
11     The pipeline engine (healthcheck -> trajectory -> windows -> AI handoff -> DB)
12     lives in trajectory_hybrid.TrajectoryHybridSegmenter; this class only wires the
13     WASB runner and identity.
14     """
15     from typing import Any, Dict, Tuple
16
17     from backend.pipeline.segmenters.base import segmenter_registry
18     from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
19     from backend.pipeline.detectors.base import DetectorRunner
20     from backend.pipeline.detectors.wasb_runner import WasbRunner, WasbConfig
21
22
23     class WasbHybridSegmenter(TrajectoryHybridSegmenter):
24         family = "wasb"
25         display_label = "WASB"
26
27         @property
28         def name(self) -> str:
29             return "wasb_hybrid"
30
31         @property
32         def description(self) -> str:
33             return ("Two-stage pipeline: WASB shuttle-trajectory detection (WSL2, pretrained
badminton "
34                     "weights) for precise candidate rallies + AI provider for semantic
boundaries.")
35
36         def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
Any]]:
37             cfg = WasbConfig.from_indexing_cfg(idx_cfg)
38             return WasbRunner(cfg), {"weights_path": cfg.weights_path}
39
40
41     segmenter_registry.register(WasbHybridSegmenter)
42

```

9. user (2026-06-13T18:45:12.299Z)

```

1      """TrackNet-based hybrid play segmenter.
2
3      Phase 2 (candidate generation) uses a TrackNetV4 *shuttle* trajectory computed
4      in WSL2 (backend/pipeline/detectors/tracknet_runner.py, ADR-001). Rationale: a
5      tiny, fast, motion-blurred shuttle is exactly what general detectors miss; a
6      purpose-built trajectory model localises it far more precisely, giving cleaner
7      candidate rallies than person-velocity heuristics.
8
9      The pipeline engine (healthcheck -> trajectory -> windows -> AI handoff -> DB)
10     lives in trajectory_hybrid.TrajectoryHybridSegmenter; this class only wires the
11     TrackNet runner and identity.
12     """
13     from typing import Any, Dict, Tuple
14
15     from backend.pipeline.segmenters.base import segmenter_registry
16     from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
17     from backend.pipeline.detectors.base import DetectorRunner
18     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner, TrackNetConfig
19
20

```

```

21     class TrackNetHybridSegmenter(TrajectoryHybridSegmenter):
22         family = "tracknet"
23         display_label = "TrackNet"
24
25         @property
26         def name(self) -> str:
27             return "tracknet_hybrid"
28
29         @property
30         def description(self) -> str:
31             return ("Two-stage pipeline: TrackNetV4 shuttle-trajectory detection (WSL2) for
precise "
32                     "candidate rallies + AI provider for high-precision semantic
boundaries.")
33
34         def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
Any]]:
35             cfg = TrackNetConfig.from_indexing_cfg(idx_cfg)
36             return TrackNetRunner(cfg), {
37                 "weights_path": cfg.weights_path,
38                 "queue_length": cfg.queue_length,
39             }
40
41
42     segmenter_registry.register(TrackNetHybridSegmenter)
43

```

10. assistant (2026-06-13T18:45:14.036Z)

Now let me read the actual runner implementations to understand parse_trajectory_csv and trajectory_to_action_windows:

11. user (2026-06-13T18:45:14.654Z)

```

1     """TrackNet shuttle-trajectory runner (WSL2 subprocess adapter).
2
3     This bridges the Windows-side indexer to a TensorFlow TrackNetV4 install living
4     inside WSL2 (ADR-001 in docs/DECISIONS.md). The flow is:
5
6         1. Translate the Windows video path to its WSL-visible form (and optionally
7            stage it into the WSL filesystem to avoid slow /mnt/c per-frame reads).
8         2. Invoke ``src/predict.py`` inside the TrackNetV4 conda env via ``wsl``.
9         3. Read back the per-frame ``Frame,Visibility,X,Y`` CSV it writes.
10        4. Convert that trajectory into high-motion "action windows" ? the same
11           candidate-rally shape HybridYoloSegmenter produces ? so the rest of the
12           pipeline (AI handoff, DB population) is unchanged.
13
14     Nothing here imports TensorFlow; all model code stays in WSL.
15     """
16
17     import os
18     import csv
19     import shlex
20     import logging
21     import subprocess
22     from dataclasses import dataclass
23     from typing import List, Dict, Any, Optional, Tuple
24
25     from backend.pipeline.detectors.base import DetectorRunner, WslCommandMixin,
wsl_tilde_quote
26
27     logger = logging.getLogger(__name__)
28
29
30     @dataclass

```

```

31     class TrackNetConfig:
32         """Resolved settings for a TrackNet WSL run (built from config.json ->
indexing.tracknet)."""
33         distro: str = "Ubuntu"
34         repo_dir: str = "~/models/TrackNetV4"          # WSL path to the cloned repo
35         conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
36         conda_env: str = "TrackNetV4"
37         weights_path: str = ""                        # WSL path to the .h5/.keras weights
(REQUIRED)
38         queue_length: int = 5
39         stage_in_wsl: bool = True                     # copy video into WSL fs first
(perf)
40         wsl_stage_dir: str = "~/clips"                # where staged videos/outputs live
in WSL
41         timeout_sec: int = 1800                      # 30 min; kills a hung call (0 = no
timeout)
42         keep_frames: bool = False                    # retain staged video after success
(debug only)
43         min_free_gb: float = 0.0                     # warn if WSL free space below this
(0 = off)
44
45     @classmethod
46     def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "TrackNetConfig":
47         tn = (idx_cfg or {}).get("tracknet", {}) or {}
48         return cls(
49             distro=tn.get("wsl_distro", "Ubuntu"),
50             repo_dir=tn.get("repo_dir", "~/models/TrackNetV4"),
51             conda_sh=tn.get("conda_sh", "~/miniconda3/etc/profile.d/conda.sh"),
52             conda_env=tn.get("conda_env", "TrackNetV4"),
53             weights_path=tn.get("weights_path", ""),
54             queue_length=int(tn.get("queue_length", 5)),
55             stage_in_wsl=bool(tn.get("stage_in_wsl", True)),
56             wsl_stage_dir=tn.get("wsl_stage_dir", "~/clips"),
57             timeout_sec=int(tn.get("timeout_sec", 1800)),
58             keep_frames=bool(tn.get("keep_frames", False)),
59             min_free_gb=float(tn.get("min_free_gb", 0.0)),
60         )
61
62
63     def to_wsl_mnt_path(win_path: str) -> str:
64         """Translate a Windows path (C:\\Users\\x) to its WSL /mnt form (/mnt/c/Users/x).
65
66         Already-POSIX paths (starting with / or ~) are returned unchanged so the
67         same helper is safe to call on either kind of path.
68         """
69         p = str(win_path)
70         if p.startswith("/") or p.startswith("~"):
71             return p
72         p = p.replace("\\", "/")
73         if len(p) >= 2 and p[1] == ":":
74             drive = p[0].lower()
75             return f"/mnt/{drive}{p[2:]}"
76         return p
77
78
79     @dataclass
80     class TrajectoryPoint:
81         frame: int
82         visible: bool
83         x: float
84         y: float
85
86
87     class TrackNetRunner(WSLCommandMixin, DetectorRunner):
88         """Runs TrackNet predict.py in WSL and turns the output CSV into action windows.

```

```

89
90 Implements the common DetectorRunner contract so a future Linux-native or
91 alternative trajectory runner can be substituted without touching the hybrids.
92 """
93
94 def __init__(self, cfg: TrackNetConfig):
95     self.cfg = cfg
96
97 def healthcheck(self) -> Tuple[bool, str]:
98     """Verify the WSL env is usable: conda env exists, TF imports, GPU visible.
99
100     Returns (ok, message). Cheap to call before a long run.
101     """
102     cmd = (
103         f"conda activate {self.cfg.conda_env} && "
104         f"python -c \"import tensorflow as tf; "
105         f"print('TF', tf.__version__, 'GPUs', "
106         f"len(tf.config.list_physical_devices('GPU')))\\"
107     )
108     try:
109         res = self._wsl_bash(cmd)
110     except FileNotFoundError:
111         return False, "`wsl` not found ? is this running on Windows with WSL2
112         installed?"
113     except subprocess.TimeoutExpired:
114         return False, "WSL healthcheck timed out."
115     out = (res.stdout or "").strip()
116     if res.returncode != 0:
117         return False, f"WSL env check failed: {(res.stderr or out).strip()}"
118     return True, out
119
120 # ----- inference
121
122 def run_predict(self, video_win_path: str, output_win_dir: str,
123                 video_id: Optional[str] = None) -> Optional[str]:
124     """Run predict.py on a video. Returns the Windows path to the produced CSV, or
125     None.
126
127     ``output_win_dir`` is a Windows directory (under the repo's output/) that
128     WSL writes into via its /mnt view, so the windows process can read the CSV
129     back with no copy. ``video_id`` (file MD5) namespaces the staged video ? and
130     therefore predict.py's ``<stem>_predict.csv`` output ? so two videos that share
131     a filename cannot collide on the output CSV.
132     """
133     if not self.cfg.weights_path:
134         logger.error(
135             "TrackNet weights_path is not set. Set indexing.tracknet.weights_path "
136             "in config.json to the WSL path of your trained TrackNetV4 weights."
137         )
138     return None
139
140 # Resolve relative dirs BEFORE the /mnt translation ? to_wsl_mnt_path passes
141 # relative paths through unchanged, so WSL would resolve them against the
142 # predict.py cwd instead of the repo (same bug class as wasb_runner).
143 output_win_dir = os.path.abspath(output_win_dir)
144 os.makedirs(output_win_dir, exist_ok=True)
145 # Normalize separators so basename/splitext work when tests (or collector)
146 # pass windows-style paths on a Linux host.
147 video_win_path = str(video_win_path).replace("\\", "/")
148 base = os.path.basename(video_win_path)
149 stem, ext = os.path.splitext(base)
150
151 # predict.py only accepts .avi/.mp4 and names the CSV "<stem>_predict.csv".
152 if ext.lower() not in (".mp4", ".avi"):
153     logger.error("TrackNet predict.py only supports .mp4/.avi (got %s)", ext)

```

```

151         return None
152
153     wsl_out = to_wsl_mnt_path(output_win_dir)
154     # Namespace by video_id so two same-filename videos don't collide on the CSV.
155     # predict.py derives its output name from the (staged) video stem, so staging
156     # under the namespaced name propagates into "<key>_predict.csv".
157     key = f"{stem}_{video_id[:12]}" if video_id else stem
158     expected_csv_win = os.path.join(output_win_dir, f"{key}_predict.csv")
159
160     # Resolve the video path WSL will read.
161     if self.cfg.stage_in_wsl:
162         staged = self._stage_video(video_win_path, f"{key}{ext}")
163         if staged is None:
164             return None
165         wsl_video = staged
166     else:
167         # No staging: predict.py reads /mnt directly and writes
168         "<stem>_predict.csv".
169         # We cannot rename its output, so fall back to the un-namespaced name.
170         wsl_video = to_wsl_mnt_path(video_win_path)
171         expected_csv_win = os.path.join(output_win_dir, f"{stem}_predict.csv")
172
173     if self.cfg.min_free_gb > 0:
174         free = self._wsl_free_gb(self.cfg.wsl_stage_dir)
175         if free is not None and free < self.cfg.min_free_gb:
176             logger.warning("Low WSL disk: %.1f GB free at %s (< min_free_gb=%.1f); "
177                             "consider running tools/cleanup_caches.py.",
178                             free, self.cfg.wsl_stage_dir, self.cfg.min_free_gb)
179
180     cmd = (
181         f"conda activate {self.cfg.conda_env} && "
182         f"cd {self.cfg.repo_dir}/src && "
183         f"python predict.py "
184         f"--video_path {wsl_tilde_quote(wsl_video)} "
185         f"--model_weights {wsl_tilde_quote(self.cfg.weights_path)} "
186         f"--output_dir {wsl_tilde_quote(wsl_out)} "
187         f"--queue_length {self.cfg.queue_length}"
188     )
189     logger.info("Running TrackNet inference on %s ...", base)
190     try:
191         res = self._wsl_bash(cmd)
192     except subprocess.TimeoutExpired:
193         logger.error("TrackNet inference timed out after %ss.",
194                     self.cfg.timeout_sec)
195         return None
196
197     if res.returncode != 0:
198         logger.error("TrackNet predict.py failed:\n%s", (res.stderr or
199 res.stdout)[-2000:])
200         return None
201
202     if not os.path.exists(expected_csv_win):
203         logger.error("TrackNet finished but expected CSV not found at %s",
204                     expected_csv_win)
205         return None
206     logger.info("TrackNet trajectory CSV: %s", expected_csv_win)
207
208     # Success ? the CSV is the durable output; drop the staged video copy (a pure,
209     # regenerable intermediate). On earlier failure we return above without
210     cleaning.
211
212     if self.cfg.stage_in_wsl and not self.cfg.keep_frames:
213         logger.info("Cache hygiene: removing staged video for %s "
214                     "(set indexing.tracknet.keep_frames=true to retain).", key)
215         self._wsl_rm_rf([wsl_video])
216     return expected_csv_win

```

```

211
212     def _stage_video(self, video_win_path: str, base: str) -> Optional[str]:
213         """Copy the input video into the WSL filesystem (avoids slow /mnt/c reads).
214
215         Returns the WSL path to the staged copy, or None on failure.
216         """
217         src_mnt = to_wsl_mnt_path(video_win_path)
218         dest = f"{self.cfg.wsl_stage_dir}/{base}"
219         cmd = f"mkdir -p {self.cfg.wsl_stage_dir} && cp {shlex.quote(src_mnt)}
{wsl_tilde_quote(dest)} && echo OK"
220         try:
221             res = self._wsl_bash(cmd)
222         except subprocess.TimeoutExpired:
223             logger.error("Staging video into WSL timed out.")
224             return None
225         if res.returncode != 0 or "OK" not in (res.stdout or ""):
226             logger.error("Failed to stage video into WSL: %s", (res.stderr or
res.stdout).strip())
227             return None
228         return dest
229
230     # ----- CSV -> windows
231
232     @staticmethod
233     def parse_trajectory_csv(csv_win_path: str) -> List[TrajectoryPoint]:
234         """Parse a TrackNet predict CSV (columns: Frame,Visibility,X,Y)."""
235         points: List[TrajectoryPoint] = []
236         with open(csv_win_path, newline="") as f:
237             reader = csv.DictReader(f)
238             for row in reader:
239                 try:
240                     points.append(TrajectoryPoint(
241                         frame=int(row["Frame"]),
242                         visible=int(row["Visibility"]) == 1,
243                         x=float(row["X"]),
244                         y=float(row["Y"]),
245                     ))
246                 except (KeyError, ValueError):
247                     continue
248             points.sort(key=lambda p: p.frame)
249             return points
250
251     @staticmethod
252     def trajectory_to_action_windows(
253         points: List[TrajectoryPoint],
254         fps: float,
255         frame_width: float,
256         chunk_start: float = 0.0,
257         velocity_thresh: float = 0.02,
258         merge_gap: float = 2.0,
259         min_window_duration: float = 2.0,
260         chunk_index: Optional[int] = None,
261     ) -> List[Dict[str, Any]]:
262         """Convert a shuttle trajectory into candidate rally windows.
263
264         A frame is "active" when the shuttle is visible AND its inter-frame
265         displacement (normalised by frame width, per second) exceeds
266         ``velocity_thresh`` ? i.e. the shuttle is in flight. Active frames within
267         ``merge_gap`` seconds of each other are grouped into one window; short
268         windows are dropped. Mirrors HybridYoloSegmenter's windowing so the dict
269         shape feeding the AI-handoff phase is identical.
270         """
271         if fps <= 0:
272             fps = 30.0
273         if frame_width <= 0:

```

```

274         frame_width = 1280.0
275
276     # Compute per-frame normalised velocity from the last *visible* point.
277     active = [] # (frame_idx, velocity)
278     prev: Optional[TrajectoryPoint] = None
279     for p in points:
280         if not p.visible:
281             prev = None # break the trajectory; absent shuttle = not in flight
282             continue
283         if prev is not None:
284             dframes = p.frame - prev.frame
285             if dframes > 0:
286                 dist = ((p.x - prev.x) ** 2 + (p.y - prev.y) ** 2) ** 0.5
287                 velocity = (dist / frame_width) * (fps / dframes)
288                 if velocity > velocity_thresh:
289                     active.append((p.frame, velocity))
290             prev = p
291
292     if not active:
293         return []
294
295     max_gap_frames = int(merge_gap * fps)
296
297     def _finalize(group: List[Tuple[int, float]]) -> Dict[str, Any]:
298         vels = [v for _, v in group]
299         return {
300             "start": group[0][0] / fps + chunk_start,
301             "end": group[-1][0] / fps + chunk_start,
302             "active_frame_count": len(group),
303             "peak_velocity": max(vels),
304             "mean_velocity": sum(vels) / len(vels),
305             "track_id_count": 1, # single shuttle; kept for window-shape parity
306             "chunk_index": chunk_index,
307         }
308
309     windows = []
310     group = [active[0]]
311     for af in active[1:]:
312         if af[0] - group[-1][0] <= max_gap_frames:
313             group.append(af)
314         else:
315             windows.append(_finalize(group))
316             group = [af]
317     windows.append(_finalize(group))
318
319     return [w for w in windows if (w["end"] - w["start"]) >= min_window_duration]
320

```

12. user (2026-06-13T18:45:14.712Z)

```

1     """WASB shuttle-trajectory runner (WSL2 subprocess adapter).
2
3     Windows-side counterpart to backend/detectors/wasb_infer.py. It:
4     1. stages the video into the WSL filesystem (fast local reads, not /mnt),
5     2. copies wasb_infer.py into the WASB repo's src/ dir,
6     3. runs it in the WSL `wasb` conda env -> a Frame,Visibility,X,Y trajectory CSV,
7     4. reuses the (model-agnostic) trajectory->action-window logic from tracknet_runner.
8
9     So the only WASB-specific bits live here; the windowing + CSV parsing are shared
10    with the TrackNet path. Verified inference core: wasb_infer.py (see its docstring).
11    """
12    import os
13    import shlex
14    import logging
15    import subprocess

```

```

16     from dataclasses import dataclass
17     from typing import Dict, Any, Optional, Tuple
18
19     # Reuse the model-agnostic helpers ? trajectory shape + windowing are identical.
20     # parse_trajectory_csv / trajectory_to_action_windows are @staticmethods on
TrackNetRunner.
21     from backend.pipeline.detectors.tracknet_runner import to_wsl_mnt_path, TrackNetRunner
22     from backend.pipeline.detectors.base import DetectorRunner, WslCommandMixin,
wsl_tilde_quote
23
24     logger = logging.getLogger(__name__)
25
26     # Windows path to the inference wrapper we stage into WSL.
27     _INFER_WIN = os.path.join(os.path.dirname(os.path.abspath(__file__)), "wasb_infer.py")
28
29
30     @dataclass
31     class WasbConfig:
32         """Resolved settings for a WASB WSL run (built from config.json ->
indexing.wasb)."""
33         distro: str = "Ubuntu"
34         repo_dir: str = "~/models/WASB-SBDT"
35         conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
36         conda_env: str = "wasb"
37         weights_path: str = "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
38         sport: str = "badminton"
39         wsl_stage_dir: str = "~/clips"
40         timeout_sec: int = 1800 # 30 min; a hung GPU/conda call is killed (0 = no timeout)
41         keep_frames: bool = False # retain staged video + frame cache after success (debug
only)
42         min_free_gb: float = 0.0 # warn if WSL free space below this before a run (0 =
off)
43
44     @classmethod
45     def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "WasbConfig":
46         w = (idx_cfg or {}).get("wasb", {}) or {}
47         return cls(
48             distro=w.get("wsl_distro", "Ubuntu"),
49             repo_dir=w.get("repo_dir", "~/models/WASB-SBDT"),
50             conda_sh=w.get("conda_sh", "~/miniconda3/etc/profile.d/conda.sh"),
51             conda_env=w.get("conda_env", "wasb"),
52             weights_path=w.get("weights_path",
53                               "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"),
54             sport=w.get("sport", "badminton"),
55             wsl_stage_dir=w.get("wsl_stage_dir", "~/clips"),
56             timeout_sec=int(w.get("timeout_sec", 1800)),
57             keep_frames=bool(w.get("keep_frames", False)),
58             min_free_gb=float(w.get("min_free_gb", 0.0)),
59         )
60
61
62     class WasbRunner(WslCommandMixin, DetectorRunner):
63         def __init__(self, cfg: WasbConfig):
64             self.cfg = cfg
65
66         def healthcheck(self) -> Tuple[bool, str]:
67             """Verify the WSL `wasb` env imports torch and sees a GPU. Returns (ok, msg)."""
68             cmd = (f"conda activate {self.cfg.conda_env} && "
69                   f"python -c \"import torch; print('torch', torch.__version__, "
70                   f"'\"'cuda', torch.cuda.is_available())\"")
71             try:
72                 res = self._wsl_bash(cmd)
73             except FileNotFoundError:

```



```

74         return False, "`wsl` not found ? Windows + WSL2 required."
75     except subprocess.TimeoutExpired:
76         return False, "WSL healthcheck timed out."
77     out = (res.stdout or "").strip()
78     if res.returncode != 0:
79         return False, f"WSL `wasb` env check failed: {(res.stderr or out).strip()}"
80     return True, out
81
82     def _stage_video(self, video_win_path: str, dest_name: str) -> Optional[str]:
83         src_mnt = to_wsl_mnt_path(video_win_path)
84         dest = f"{self.cfg.wsl_stage_dir}/{dest_name}"
85         cmd = f"mkdir -p {self.cfg.wsl_stage_dir} && cp {shlex.quote(src_mnt)}"
86         {wsl_tilde_quote(dest)} && echo OK"
87         try:
88             res = self._wsl_bash(cmd)
89         except subprocess.TimeoutExpired:
90             logger.error("Staging video into WSL timed out.")
91             return None
92         if res.returncode != 0 or "OK" not in (res.stdout or ""):
93             logger.error("Failed to stage video into WSL: %s", (res.stderr or
94 res.stdout).strip())
95             return None
96         return dest
97
98     def _stage_infer_script(self) -> bool:
99         """Copy wasb_infer.py into the WASB repo src/ so it can import WASB modules."""
100         src_mnt = to_wsl_mnt_path(_INFER_WIN)
101         cmd = f"cp {shlex.quote(src_mnt)} {self.cfg.repo_dir}/src/wasb_infer.py && echo
102 OK"
103         res = self._wsl_bash(cmd)
104         return res.returncode == 0 and "OK" in (res.stdout or "")
105
106     def run_predict(self, video_win_path: str, output_win_dir: str,
107                     video_id: Optional[str] = None) -> Optional[str]:
108         """Run WASB on a video. Returns the Windows path to the trajectory CSV, or None.
109
110         All WSL/cache/output artifacts are namespaced by ``video_id`` (the file MD5) so
111         two videos that share a filename (e.g. two ``match.mp4`` from different folders)
112         can never reuse each other's staged frames, resumable cache, or CSV.
113
114         # Resolve relative dirs (the hybrids pass "output/wasb") BEFORE the /mnt
115         # translation: to_wsl_mnt_path passes relative paths through unchanged, so
116         # WSL resolved them against the inference cwd (~/models/WASB-SBDT/src) and
117         # the CSV landed inside WSL while Windows polled the repo-relative path.
118         output_win_dir = os.path.abspath(output_win_dir)
119         os.makedirs(output_win_dir, exist_ok=True)
120         # Normalize for cross-platform basename (tests use Windows paths on Linux).
121         video_win_path = str(video_win_path).replace("\\", "/")
122         base = os.path.basename(video_win_path)
123         stem, ext = os.path.splitext(base)
124         # Unique, content-derived key: same filename + different bytes -> different key.
125         key = f"{stem}__{video_id[:12]}" if video_id else stem
126         wsl_out_dir = to_wsl_mnt_path(output_win_dir)
127         expected_csv_win = os.path.join(output_win_dir, f"{key}_wasb.csv")
128
129         if not self._stage_infer_script():
130             logger.error("Could not stage wasb_infer.py into WSL.")
131             return None
132         staged_video = self._stage_video(video_win_path, f"{key}{ext}")
133         if staged_video is None:
134             return None
135         frames_dir = f"{self.cfg.wsl_stage_dir}/{key}_frames"
136
137         # Disk guard: a 4K clip can extract ~100 GB of PNG frames. Warn early if the
138         # WSL stage filesystem is already low so the user can clean up before it fills.

```

```

136         if self.cfg.min_free_gb > 0:
137             free = self._wsl_free_gb(self.cfg.wsl_stage_dir)
138             if free is not None and free < self.cfg.min_free_gb:
139                 logger.warning(
140                     "Low WSL disk: %.1f GB free at %s (< min_free_gb=%.1f). Frame
extraction "
141                     "may fill the disk ? consider running tools/cleanup_caches.py.",
142                     free, self.cfg.wsl_stage_dir, self.cfg.min_free_gb)
143
144         cmd = (
145             f"conda activate {self.cfg.conda_env} && cd {self.cfg.repo_dir}/src && "
146             f"python wasb_infer.py "
147             f"--video {wsl_tilde_quote(staged_video)} "
148             f"--frames_out_dir {wsl_tilde_quote(frames_dir)} "
149             f"--weights {wsl_tilde_quote(self.cfg.weights_path)} "
150             f"--sport {shlex.quote(self.cfg.sport)} "
151             f"--out {wsl_tilde_quote(wsl_out_dir + '/' + key + '_wasb.csv')}}")
152         )
153         logger.info("Running WASB inference on %s ...", base)
154         try:
155             res = self._wsl_bash(cmd)
156         except subprocess.TimeoutExpired:
157             logger.error("WASB inference timed out after %ss.", self.cfg.timeout_sec)
158             return None
159         if res.returncode != 0:
160             logger.error("WASB inference failed:\n%s", (res.stderr or
res.stdout)[-2000:])
161             return None
162         if not os.path.exists(expected_csv_win):
163             logger.error("WASB finished but expected CSV not found at %s",
expected_csv_win)
164             return None
165         logger.info("WASB trajectory CSV: %s", expected_csv_win)
166
167         # Success ? the trajectory CSV is the durable output; the staged video copy and
168         # the (huge) decoded frame cache are now pure intermediates. Delete them unless
169         # explicitly retained. On any earlier failure we return above WITHOUT cleaning,
170         # so the frame cache + detector cache remain for a resumable re-run.
171         if not self.cfg.keep_frames:
172             logger.info("Cache hygiene: removing staged video + frame cache for %s "
173                         "(set indexing.wasb.keep_frames=true to retain).", key)
174             self._wsl_rm_rf([frames_dir, staged_video])
175         return expected_csv_win
176
177         # Trajectory parsing + windowing are inherited from tracknet_runner (model-
agnostic).
178         parse_trajectory_csv = staticmethod(TrackNetRunner.parse_trajectory_csv)
179         trajectory_to_action_windows =
staticmethod(TrackNetRunner.trajectory_to_action_windows)
180

```

13. assistant (2026-06-13T18:45:16.600Z)

Perfect. Now let me check the persistence call and the add_candidate_segment method to map the stats schema:

14. user (2026-06-13T18:45:17.265Z)

Found 10 files
tests\test_yolo_hybrid.py
backend\pipeline\segmenters\yolo_hybrid.py
tests\test_experiment.py
backend\pipeline\segmenters\trajectory_hybrid.py
tests\test_trajectory_hybrid_equivalence.py
tests\test_tracknet_hybrid.py

```
backend\infrastructure\database.py
tests\test_database.py
tests\test_reingest_safety.py
tests\test_eval_harness.py
```

15. user (2026-06-13T18:45:18.524Z)

```
1     import sqlite3
2     import json
3     import logging
4     from contextlib import contextmanager
5     from typing import List, Dict, Any, Optional, Tuple
6
7     logger = logging.getLogger(__name__)
8
9     class Database:
10         def __init__(self, db_path: str):
11             self.db_path = db_path
12             self._init_db()
13
14         @contextmanager
15         def _connect(self):
16             """Transaction scope that also CLOSES the connection.
17
18             ``with sqlite3.connect(...)`` alone is only a commit/rollback scope ? it
19             never closes, so every call leaked its connection to GC (lingering file
20             handles on the WAL sidecars under Windows). Internal methods use this.
21             """
22             conn = self._get_connection()
23             try:
24                 with conn:
25                     yield conn
26             finally:
27                 conn.close()
28
29         def _get_connection(self):
30             # busy_timeout: wait (not fail) on a locked DB ? needed once the async job model
31             # adds concurrent writers (a swallowed 'database is locked' silently drops
32             segments).
33             conn = sqlite3.connect(self.db_path, timeout=30.0)
34             conn.row_factory = sqlite3.Row
35             # Enforce ON DELETE CASCADE / SET NULL ? SQLite leaves FKs off per-connection.
36             conn.execute("PRAGMA foreign_keys = ON")
37             # WAL allows concurrent readers during a write; busy_timeout bounds lock waits.
38             conn.execute("PRAGMA busy_timeout = 30000")
39             try:
40                 conn.execute("PRAGMA journal_mode = WAL")
41             except sqlite3.OperationalError:
42                 pass # e.g. a read-only/networked FS that can't do WAL ? degrade gracefully
43             return conn
44
45         def _init_db(self):
46             """Initializes tables for videos, play_segments, and events."""
47             with self._connect() as conn:
48                 cursor = conn.cursor()
49
50                 # Videos Table
51                 cursor.execute("""
52                     CREATE TABLE IF NOT EXISTS videos (
53                         id TEXT PRIMARY KEY,
54                         filepath TEXT NOT NULL,
55                         processed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
56                         status TEXT NOT NULL,
57                         validation_results TEXT
58                     )
59                 """)
```

```

58         """)
59
60     # Play Segments Table (Extensible metadata JSON)
61     cursor.execute("""
62         CREATE TABLE IF NOT EXISTS play_segments (
63             id INTEGER PRIMARY KEY AUTOINCREMENT,
64             video_id TEXT NOT NULL,
65             start_time REAL NOT NULL,
66             end_time REAL NOT NULL,
67             duration REAL NOT NULL,
68             server TEXT,
69             receiver TEXT,
70             metadata TEXT,
71             FOREIGN KEY (video_id) REFERENCES videos(id) ON DELETE CASCADE
72         )
73     """)
74
75     # Events Table
76     cursor.execute("""
77         CREATE TABLE IF NOT EXISTS events (
78             id INTEGER PRIMARY KEY AUTOINCREMENT,
79             segment_id INTEGER NOT NULL,
80             timestamp REAL NOT NULL,
81             type TEXT NOT NULL,
82             player TEXT,
83             details TEXT,
84             FOREIGN KEY (segment_id) REFERENCES play_segments(id) ON DELETE
85         )
86     """)
87
88     # Versioned migrations live here. PRAGMA user_version is the schema
89     # contract ? bump it for every change and add an ordered `if version < N`
90     # block below. Tests assert the expected version, so accidental drift fails
91     CI.
92     version = cursor.execute("PRAGMA user_version").fetchone()[0]
93
94     if version < 1:
95         # v1: raw candidate detections (pre-confirmation), detector-agnostic.
96         # Common fields are columns; detector-specific metrics go in `stats`
97         JSON,
98         # so a new segmenter reuses this table by setting its own
99         `detector`/`stats`.
100         cursor.execute("""
101             CREATE TABLE IF NOT EXISTS candidate_segments (
102                 id INTEGER PRIMARY KEY AUTOINCREMENT,
103                 video_id TEXT NOT NULL,
104                 detector TEXT NOT NULL,
105                 chunk_index INTEGER,
106                 start_time REAL NOT NULL,
107                 end_time REAL NOT NULL,
108                 duration REAL NOT NULL,
109                 confidence REAL,
110                 stats TEXT,
111                 detection_params TEXT,
112                 confirmed_segment_id INTEGER,
113                 human_label TEXT,
114                 notes TEXT,
115                 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
116                 FOREIGN KEY (video_id) REFERENCES videos(id) ON DELETE CASCADE,
117                 FOREIGN KEY (confirmed_segment_id) REFERENCES play_segments(id)
118             )
119         """)
120         cursor.execute("""

```

```

118             CREATE INDEX IF NOT EXISTS idx_candidates_video_detector
119             ON candidate_segments(video_id, detector)
120         """)
121         cursor.execute("PRAGMA user_version = 1")
122
123     if version < 2:
124         # v2: async job model (DEFERRED_HARDENING_PLAN #16). One row per
submitted
125         # /api/process request. `status` is the EXECUTION state of the job
126         # (queued|running|done|failed); the pipeline's own verdict (e.g. a video
127         # that failed validation) lives inside `result` JSON as
result["status"].
128         # `result` is the full sync-era response dict (incl. report paths), so
the
129         # observability report is the job's artifact, per the plan.
130         cursor.execute("""
131             CREATE TABLE IF NOT EXISTS jobs (
132                 id TEXT PRIMARY KEY,
133                 video_path TEXT NOT NULL,
134                 video_id TEXT,
135                 segmenter TEXT,
136                 player_pool TEXT,
137                 max_frames INTEGER,
138                 status TEXT NOT NULL DEFAULT 'queued',
139                 progress TEXT,
140                 error TEXT,
141                 result TEXT,
142                 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
143                 started_at TIMESTAMP,
144                 finished_at TIMESTAMP
145             )
146         """)
147         cursor.execute("CREATE INDEX IF NOT EXISTS idx_jobs_status ON
jobs(status)")
148         cursor.execute("PRAGMA user_version = 2")
149         # future: if version < 3: cursor.execute("ALTER TABLE ..."); PRAGMA
user_version = 3
150
151         conn.commit()
152
153     def add_video(self, video_id: str, filepath: str, status: str, validation_results:
List[Dict[str, Any]]) -> bool:
154         """Register or update a video row WITHOUT touching its child segments.
155
156         Uses UPSERT, not INSERT OR REPLACE: REPLACE deletes the conflicting row, which ?
157         with foreign_keys ON ? cascade-deletes every
play_segment/event/candidate_segment.
158         That made a status update (e.g. marking 'failed' on re-validation, or
'processed'
159         before the segmenter runs) silently destroy prior good results. UPSERT mutates
the
160         row in place; clearing segments is now an explicit, deliberate operation
161         (clear_video_data / prune_segments).
162         """
163         try:
164             with self._connect() as conn:
165                 conn.cursor().execute(
166                     "INSERT INTO videos (id, filepath, status, validation_results)
VALUES (?, ?, ?, ?) "
167                     "ON CONFLICT(id) DO UPDATE SET "
168                     "filepath=excluded.filepath, status=excluded.status, "
169                     "validation_results=excluded.validation_results",
170                     (video_id, filepath, status, json.dumps(validation_results))
171                 )
172             conn.commit()

```

```

173         return True
174     except Exception as e:
175         logger.error(f"Failed to add video: {e}")
176         return False
177
178     def segment_watermarks(self, video_id: str) -> Tuple[int, int]:
179         """Return (max play_segment id, max candidate_segment id) for a video (0 if
180         none).
181         Captured before a (re)segmentation so the run's new rows (id > watermark) can be
182         told apart from prior rows (id <= watermark), enabling destroy-AFTER-confirm.
183         """
184         with self._connect() as conn:
185             cur = conn.cursor()
186             p = cur.execute("SELECT COALESCE(MAX(id), 0) FROM play_segments WHERE
187             video_id = ?",
188                             (video_id,)).fetchone()[0]
189             c = cur.execute("SELECT COALESCE(MAX(id), 0) FROM candidate_segments WHERE
190             video_id = ?",
191                             (video_id,)).fetchone()[0]
192             return int(p), int(c)
193
194     def prune_segments(self, video_id: str, *, play_upto: Optional[int] = None,
195                       play_after: Optional[int] = None, cand_upto: Optional[int] =
196                       None,
197                       cand_after: Optional[int] = None) -> None:
198         """Delete play/candidate segments by id range (events cascade from
199         play_segments).
200         Used to finish a (re)segmentation: keep the NEW rows and drop the OLD ones on a
201         successful run (`*_upto` = the pre-run watermark), or drop the NEW partial
202         rows
203         and keep the OLD ones when the run failed/produced nothing (`*_after`).
204         """
205         with self._connect() as conn:
206             cur = conn.cursor()
207             if play_upto is not None:
208                 cur.execute("DELETE FROM play_segments WHERE video_id = ? AND id <= ?",
209                             (video_id, play_upto))
210             if play_after is not None:
211                 cur.execute("DELETE FROM play_segments WHERE video_id = ? AND id > ?",
212                             (video_id, play_after))
213             if cand_upto is not None:
214                 cur.execute("DELETE FROM candidate_segments WHERE video_id = ? AND id <=
215                 ?", (video_id, cand_upto))
216             if cand_after is not None:
217                 cur.execute("DELETE FROM candidate_segments WHERE video_id = ? AND id >
218                 ?", (video_id, cand_after))
219             conn.commit()
220
221     def add_play_segment(self, video_id: str, start_time: float, end_time: float,
222                       server: Optional[str] = None, receiver: Optional[str] = None, metadata: Optional[Dict[str, Any]]
223                       = None) -> Optional[int]:
224         try:
225             with self._connect() as conn:
226                 cursor = conn.cursor()
227                 cursor.execute(
228                     "INSERT INTO play_segments (video_id, start_time, end_time,
229                     duration, server, receiver, metadata) VALUES (?, ?, ?, ?, ?, ?, ?)",
230                     (video_id, start_time, end_time, end_time - start_time, server,
231                     receiver, json.dumps(metadata or {})))
232                 )
233                 conn.commit()
234                 return cursor.lastrowid
235         except Exception as e:

```

```

224         logger.error(f"Failed to add play segment: {e}")
225         return None
226
227     def add_event(self, segment_id: int, timestamp: float, event_type: str, player:
Optional[str] = None, details: Optional[Dict[str, Any]] = None) -> bool:
228         try:
229             with self._connect() as conn:
230                 conn.cursor().execute(
231                     "INSERT INTO events (segment_id, timestamp, type, player, details)
VALUES (?, ?, ?, ?, ?)",
232                     (segment_id, timestamp, event_type, player, json.dumps(details or
{})))
233             )
234             conn.commit()
235             return True
236         except Exception as e:
237             logger.error(f"Failed to add event: {e}")
238             return False
239
240     def add_candidate_segment(self, video_id: str, detector: str, start_time: float,
end_time: float,
241                             chunk_index: Optional[int] = None, confidence:
Optional[float] = None,
242                             stats: Optional[Dict[str, Any]] = None,
243                             detection_params: Optional[Dict[str, Any]] = None) ->
Optional[int]:
244         """Persist one raw (pre-confirmation) detection. `stats`/`detection_params` are
245         detector-specific dicts stored as JSON. Returns the new candidate id, or
None."""
246         try:
247             with self._connect() as conn:
248                 cursor = conn.cursor()
249                 cursor.execute(
250                     """INSERT INTO candidate_segments
"""

```

16. user (2026-06-13T18:45:20.457Z)

```

240     def add_candidate_segment(self, video_id: str, detector: str, start_time: float,
end_time: float,
241                             chunk_index: Optional[int] = None, confidence:
Optional[float] = None,
242                             stats: Optional[Dict[str, Any]] = None,
243                             detection_params: Optional[Dict[str, Any]] = None) ->
Optional[int]:
244         """Persist one raw (pre-confirmation) detection. `stats`/`detection_params` are
245         detector-specific dicts stored as JSON. Returns the new candidate id, or
None."""
246         try:
247             with self._connect() as conn:
248                 cursor = conn.cursor()
249                 cursor.execute(
250                     """INSERT INTO candidate_segments
251                     (video_id, detector, chunk_index, start_time, end_time, duration,
252                     confidence, stats, detection_params)
253                     VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)""",
254                     (video_id, detector, chunk_index, start_time, end_time, end_time -
start_time,
255                     confidence, json.dumps(stats or {}), json.dumps(detection_params or
{})))
256             )
257             conn.commit()
258             return cursor.lastrowid
259         except Exception as e:
260             logger.error(f"Failed to add candidate segment: {e}")
261             return None

```

```

262
263     def link_candidate_to_segment(self, candidate_id: int, segment_id: int) -> bool:
264         """Record that a candidate detection was confirmed as a given play_segment."""
265         try:
266             with self._connect() as conn:
267                 conn.cursor().execute(
268                     "UPDATE candidate_segments SET confirmed_segment_id = ? WHERE id =
269                     ?",
270                     (segment_id, candidate_id)
271                 )
272                 conn.commit()
273             return True
274         except Exception as e:
275             logger.error(f"Failed to link candidate {candidate_id} to segment
276             {segment_id}: {e}")
277             return False
278
279     def query_candidate_segments(self, video_id: str, detector: Optional[str] = None,
280                                only_unlabeled: bool = False) -> List[Dict[str, Any]]:
281         """Fetch candidate detections for the annotation / fine-tuning workflow.
282         `stats` and `detection_params` are deserialized from JSON."""
283         query = "SELECT * FROM candidate_segments WHERE video_id = ?"
284         params: List[Any] = [video_id]
285
286         if detector:
287             query += " AND detector = ?"
288             params.append(detector)
289         if only_unlabeled:
290             query += " AND human_label IS NULL"
291
292         query += " ORDER BY start_time"
293
294         candidates = []
295         with self._connect() as conn:
296             rows = conn.cursor().execute(query, params).fetchall()
297             for row in rows:
298                 d = dict(row)
299                 d["stats"] = json.loads(d["stats"] or "{}")
300                 d["detection_params"] = json.loads(d["detection_params"] or "{}")
301                 candidates.append(d)
302         return candidates
303
304     def get_video(self, video_id: str) -> Optional[Dict[str, Any]]:
305         with self._connect() as conn:
306             row = conn.cursor().execute("SELECT * FROM videos WHERE id = ?",
307             (video_id,)).fetchone()
308             if row:
309                 d = dict(row)
310                 d["validation_results"] = json.loads(d["validation_results"] or "[]")
311                 return d
312             return None
313
314     def query_play_segments(self, video_id: str, filters: Dict[str, Any]) ->
315     List[Dict[str, Any]]:
316         """
317         Dynamically queries play segments based on filters:
318         - duration_min: float
319         - server: string
320         - receiver: string
321         - event_type: string (e.g. smash, foul)
322         """
323         query = "SELECT r.* FROM play_segments r WHERE r.video_id = ?"
324         params = [video_id]
325
326         if filters.get("duration_min"):

```



```

323         query += " AND r.duration >= ?"
324         params.append(filters["duration_min"])
325
326     if filters.get("server"):
327         query += " AND r.server = ?"
328         params.append(filters["server"])
329
330     if filters.get("receiver"):
331         query += " AND r.receiver = ?"
332         params.append(filters["receiver"])
333
334     if filters.get("event_type"):
335         # Check if there is an event of a specific type inside this segment
336         query += " AND EXISTS (SELECT 1 FROM events e WHERE e.segment_id = r.id AND
e.type = ?)"
337         params.append(filters["event_type"])
338
339     segments_list = []

```

17. assistant (2026-06-13T18:45:21.863Z)

Excellent. Now let me look at the base segmenter to understand the class contract:

18. user (2026-06-13T18:45:22.316Z)

```

1     from abc import ABC, abstractmethod
2     from typing import List, Dict, Any, Type, Optional
3     from backend.infrastructure.database import Database
4     from backend.results import Result # new standardized contract (Result[Value] ==
Success[Value] | Failure)
5
6     class BasePlaySegmenter(ABC):
7         def __init__(self, db: Database, config: Dict[str, Any]):
8             self.db = db
9             self.config = config
10
11         @property
12         @abstractmethod
13         def name(self) -> str:
14             """Unique identifier for this segmenter."""
15             pass
16
17         @property
18         @abstractmethod
19         def description(self) -> str:
20             """Human-readable explanation of this segmenter."""
21             pass
22
23         @abstractmethod
24         def process_video(self, video_id: str, video_path: str, player_pool: List[str] =
None, max_frames: int = None) -> "Result[List[Dict[str, Any]]]":
25             """
26             Processes a video to detect play segments, populates SQLite database,
27             and returns a Result.
28
29             Use isinstance(result, Failure) (or result.is_success()) to distinguish
30             "no segments found" from "something went wrong" (missing key, healthcheck
31             failure, etc.). This is the start of the standardized error/result contract.
32             """
33             pass
34
35     class SegmenterRegistry:
36         def __init__(self):
37             self._segmenters: Dict[str, Type[BasePlaySegmenter]] = {}
38

```

```

39     def register(self, segmenter_cls: Type[BasePlaySegmenter]):
40         """Registers a new segmenter class under its name property."""
41         try:
42             temp_inst = segmenter_cls(None, {})
43             name = temp_inst.name
44             except Exception as e:
45                 raise ValueError(f"Failed to resolve name property from segmenter class
{segmenter_cls.__name__}: {e}") from e
46             self._segmenters[name] = segmenter_cls
47
48     def get(self, name: str) -> Optional[Type[BasePlaySegmenter]]:
49         return self._segmenters.get(name)
50
51     def list_available(self) -> Dict[str, str]:
52         """Returns a dict of registered segmenter names and their descriptions."""
53         res = {}
54         for name, cls in self._segmenters.items():
55             try:
56                 temp_inst = cls(None, {})
57                 desc = temp_inst.description
58             except Exception:
59                 desc = "No description available."
60             res[name] = desc
61         return res
62
63     segmenter_registry = SegmenterRegistry()
64

```

19. assistant (2026-06-13T18:45:24.115Z)

Perfect. Now let me create a comprehensive map by reviewing key sequences and understanding the exact plugging points. Let me check if there's any rail_gate or person detector reference to understand the cue plugging:

20. user (2026-06-13T18:45:27.129Z)

```

1     """Rally-start gate: suppress between-point false fires using shuttle net-crossings.
2
3     WASB (shuttle-only) over-detects: when a player flicks/tosses the shuttle back to
4     the opponent for service after losing a point, that single trajectory looks like a
5     rally. The discriminator is structural, not duration-based: a real rally has the
6     shuttle crossing the net MULTIPLE times (each shot lands on the other side); a
7     single service feed crosses ~once.
8
9     This module is camera-agnostic and needs NO new model ? it works on the trajectory
10    we already produce (Frame,Visibility,X,Y). It estimates the play axis (the image
11    axis with the larger shuttle spread) and the net position (median shuttle coord on
12    that axis ? the shuttle clusters at the two ends and crosses the net region often,
13    so the median sits near the net), then counts sign changes of (coord - net) across
14    the visible points inside a candidate window, with a deadband to ignore jitter.
15
16    Gate A in the over-fire fix spectrum (B = player-stance state machine, future).
17    Pure functions only ? no I/O, unit-tested; intended as a post-filter on the
18    windows from trajectory_to_action_windows, or to fold into it later.
19    """
20    from __future__ import annotations
21
22    from dataclasses import dataclass
23    from typing import List, Sequence, Tuple
24
25    Interval = Tuple[float, float]
26
27
28    @dataclass
29    class GatedWindow:

```

```

30     start: float
31     end: float
32     crossings: int
33     visible_points: int
34     kept: bool
35
36
37 def _spread(vals: Sequence[float]) -> float:
38     """Robust spread = p95 - p5 (ignores a few outlier detections)."""
39     if len(vals) < 2:
40         return 0.0
41     s = sorted(vals)
42     lo = s[max(0, int(0.05 * (len(s) - 1)))]
43     hi = s[min(len(s) - 1, int(0.95 * (len(s) - 1)))]
44     return hi - lo
45
46
47 def _median(vals: Sequence[float]) -> float:
48     if not vals:
49         return 0.0
50     s = sorted(vals)
51     n = len(s)
52     return s[n // 2] if n % 2 else 0.5 * (s[n // 2 - 1] + s[n // 2])
53
54
55 def estimate_play_axis(points) -> Tuple[str, float, float]:
56     """Return (axis 'x'/'y', net_value, span) from visible points.
57
58     Play axis = the image axis along which the shuttle travels most (larger spread).
59     For a baseline camera that's Y (players top/bottom); for a side camera, X.
60     """
61     xs = [p.x for p in points if p.visible]
62     ys = [p.y for p in points if p.visible]
63     sx, sy = _spread(xs), _spread(ys)
64     if sx >= sy:
65         return "x", _median(xs), sx
66     return "y", _median(ys), sy
67
68
69 def count_net_crossings(window_points, axis: str, net: float, deadband: float) -> int:
70     """Count sign changes of (coord - net) across visible points, ignoring the
71     deadband zone near the net (so jitter at the net doesn't inflate the count)."""
72     crossings = 0
73     last_side = None
74     for p in window_points:
75         if not p.visible:
76             continue
77         v = (p.x if axis == "x" else p.y) - net
78         if abs(v) < deadband:
79             continue
80         side = v > 0
81         if last_side is not None and side != last_side:
82             crossings += 1
83         last_side = side
84     return crossings
85
86
87 def _points_in(points, fps: float, start: float, end: float):
88     return [p for p in points if start * fps <= p.frame <= end * fps]
89
90
91 def gate_windows(points, windows: List[Interval], fps: float,
92                 min_crossings: int = 2, deadband_frac: float = 0.06) ->
List[GatedWindow]:
93     """Annotate each (start,end) window with its shuttle net-crossing count and a

```

```

94     keep decision (crossings >= min_crossings). Deadband is a fraction of the play
95     axis span. Net/axis are estimated once over the whole trajectory (stable)."""
96     axis, net, span = estimate_play_axis(points)
97     deadband = deadband_frac * span
98     out: List[GatedWindow] = []
99     for (s, e) in windows:
100         wp = _points_in(points, fps, s, e)
101         c = count_net_crossings(wp, axis, net, deadband)
102         vis = sum(1 for p in wp if p.visible)
103         out.append(GatedWindow(start=s, end=e, crossings=c, visible_points=vis,
104                               kept=c >= min_crossings))
105     return out
106
107
108 def apply_gate(points, windows: List[Interval], fps: float,
109               min_crossings: int = 2, deadband_frac: float = 0.06) -> List[Interval]:
110     """Convenience: return only the windows that pass the crossing gate."""
111     return [(g.start, g.end) for g in
112             gate_windows(points, windows, fps, min_crossings, deadband_frac) if g.kept]
113

```

21. assistant (2026-06-13T18:45:28.498Z)

Now let me check the person_detector to see how person cue would integrate:

22. user (2026-06-13T18:45:28.898Z)

```

1     """Person-detection rally cue ? torchvision detector + ByteTrack + motion windowing.
2
3     Extracted verbatim from `yolo_hybrid` (fusion P1, `docs/MULTI_SIGNAL_FUSION_PLAN.md`):
4     the **person cue is a swappable producer**, not logic baked into one segmenter. Today
5     `HybridYoloSegmenter` is the only consumer (it delegates Stage-1 here, byte-
6     identically);
7     next the fusion segmenter consumes the SAME producer over shuttle-seeded windows, and
8     later the learned classifier reads its persisted features. Keeping detection here (not
9     in
10    the segmenter) is what makes "add a cue = register a producer" true.
11
12    Behaviour is unchanged from the in-lined version ? only the home moved:
13    - `lazy_load_model`           ? load a permissively-licensed torchvision detector
14    - `make_tracker`             ? a fresh supervision ByteTrack per chunk
15    - `detect_and_track`         ? one frame ? confident persons with persistent IDs
16    - `extract_action_windows_from_chunk` ? a chunk ? high-motion candidate windows
17    with the motion stats (`peak/mean_velocity`, `active_frame_count`, `track_id_count`)
18    that feed the candidate `stats` JSON and the learned fusion features.
19
20    Heavy deps (torchvision, supervision) are imported lazily inside the methods so
21    importing
22    the segmenter registry never pulls torch on a machine that lacks it (and tests can mock
23    the seams). Licensing: torchvision detectors are BSD + COCO weights; ByteTrack is MIT
24    (ADR-005; ultralytics' AGPL YOLO was dropped).
25    """
26    import logging
27    from typing import Any, Dict, List, Optional, Tuple
28
29    import cv2
30    import torch
31
32    from backend.utils.geometry import get_velocity_normalised
33
34    logger = logging.getLogger(__name__)
35
36    # torchvision detection models are trained on COCO where the "person" category is
37    # class 1 (index 0 is the implicit background). NB: ultralytics YOLO used class 0 ?
38    # this off-by-one is the classic gotcha when swapping detectors. Keep it named.

```

```

36     _PERSON_CLASS_ID = 1
37
38     # Permissively-licensed torchvision detectors selectable via config
39     # `indexing.detector_model`. All are BSD-licensed (code + COCO weights).
40     _DETECTOR_FACTORIES = {
41         "fasterrcnn_resnet50_fpn": "fasterrcnn_resnet50_fpn",
42         "fasterrcnn_mobilenet_v3_large_fpn": "fasterrcnn_mobilenet_v3_large_fpn",
43         "retinanet_resnet50_fpn": "retinanet_resnet50_fpn",
44     }
45     _DEFAULT_DETECTOR = "fasterrcnn_resnet50_fpn"
46
47
48     class PersonDetector:
49         """Stage-1 person detection + tracking + motion-window extraction (a rally cue).
50
51         Owns the torchvision model + per-chunk ByteTrack tracker. Constructed with the run
52         ``config`` (reads ``indexing.{use_gpu, detector_model, detector_score_threshold,
53         min_action_window_duration}``); the model loads lazily on first use.
54         """
55
56         def __init__(self, config: Dict[str, Any]):
57             self.config = config
58             # Dynamic torchvision module (None until lazy_load_model); typed Any so the
59             # `self.model([tensor])` inference call type-checks without a quarantine.
60             self.model: Any = None
61             self.device: Optional[str] = None
62
63         def lazy_load_model(self) -> None:
64             if self.model is not None:
65                 return
66
67             # Imported lazily so the module imports without torchvision present and so
68             # the test suite (which pre-sets self.model) never pulls in model weights.
69             from torchvision.models import detection as tv_detection
70
71             idx_cfg = self.config.get("indexing", {})
72             use_gpu = idx_cfg.get("use_gpu", True)
73             self.device = "cuda" if use_gpu and torch.cuda.is_available() else "cpu"
74             if use_gpu and not torch.cuda.is_available():
75                 logger.warning("use_gpu is True but CUDA is not available. Falling back to
CPU.")
76
77             model_name = idx_cfg.get("detector_model", _DEFAULT_DETECTOR)
78             if model_name not in _DETECTOR_FACTORIES:
79                 logger.warning(f"Unknown detector_model '{model_name}'. Falling back to
{_DEFAULT_DETECTOR}.")
80                 model_name = _DEFAULT_DETECTOR
81
82             builder = getattr(tv_detection, _DETECTOR_FACTORIES[model_name])
83             logger.info(f"Loading torchvision detector '{model_name}' on {self.device}...")
84             # weights="DEFAULT" pulls the best available COCO-pretrained checkpoint.
85             self.model = builder(weights="DEFAULT")
86             self.model.eval()
87             self.model.to(self.device)
88
89         def make_tracker(self, fps: float) -> Any:
90             """Create a fresh ByteTrack tracker for one chunk (track IDs are chunk-local).
91
92             Imported lazily so tests can mock this method without supervision installed.
93             """
94             import supervision as sv
95             frame_rate = max(1, int(round(fps)))
96             return sv.ByteTrack(frame_rate=frame_rate)
97
98         def detect_and_track(self, frame: Any, tracker: Any,

```

```

99         score_thresh: float) -> List[Tuple[int, Tuple[float, float,
float, float]]]:
100         """Detect persons in a frame and assign persistent track IDs.
101
102         Returns a list of (track_id, bbox) where bbox is (x1, y1, x2, y2) in pixels.
103         This replaces the old ultralytics ``model.track(...)`` call, which bundled
104         detection AND association: torchvision does the detection, supervision's
105         ByteTrack does the association.
106         """
107         import supervision as sv
108
109         # cv2 frames are BGR uint8 HxWxC; torchvision wants RGB float [0,1] CxHxW.
110         rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
111         tensor = torch.from_numpy(rgb).permute(2, 0,
1).float().div(255.0).to(self.device)
112         with torch.no_grad():
113             outputs = self.model([tensor])
114
115         out = outputs[0]
116         boxes = out["boxes"].detach().cpu().numpy()
117         labels = out["labels"].detach().cpu().numpy()
118         scores = out["scores"].detach().cpu().numpy()
119
120         # Keep only confident person detections.
121         mask = (labels == _PERSON_CLASS_ID) & (scores >= score_thresh)
122         if not mask.any():
123             return []
124
125         # ByteTrack requires confidence to be populated (it filters on it internally).
126         dets = sv.Detections(xyxy=boxes[mask], confidence=scores[mask],
class_id=labels[mask])
127         tracked = tracker.update_with_detections(dets)
128
129         result: List[Tuple[int, Tuple[float, float, float, float]]] = []
130         for i in range(len(tracked)):
131             tid = tracked.tracker_id[i]
132             if tid is None:
133                 continue
134             x1, y1, x2, y2 = tracked.xyxy[i]
135             result.append((int(tid), (float(x1), float(y1), float(x2), float(y2))))
136         return result
137
138     def extract_action_windows_from_chunk(self, chunk_path: str, chunk_start: float,
139                                         velocity_thresh: float, merge_gap: float,
140                                         frame_skip: int = 3,
141                                         chunk_index: Optional[int] = None) ->
List[Dict[str, Any]]:
142         """
143         Runs person detection + tracking on a video chunk and extracts high-motion
144         action windows. Returns a list of dicts in the full video's timeframe, each
with:
145             start, end (seconds), active_frame_count, peak_velocity, mean_velocity,
146             track_id_count, chunk_index.
147
148         Args:
149             frame_skip: Process every Nth frame (1 = every frame). Higher values
150                 dramatically reduce inference cost with minimal quality loss.
151             chunk_index: Index of the source chunk (recorded on each window for
traceability).
152         """
153         cap = cv2.VideoCapture(chunk_path)
154         if not cap.isOpened():
155             logger.error(f"Could not open {chunk_path}")
156             return []
157

```

```

158     fps = cap.get(cv2.CAP_PROP_FPS)
159     if fps <= 0:
160         fps = 30.0
161
162     frame_width = cap.get(cv2.CAP_PROP_FRAME_WIDTH)
163     if frame_width <= 0:
164         frame_width = 1280.0 # Sensible fallback
165
166     # Effective sampling rate after frame-skipping
167     effective_fps = fps / max(frame_skip, 1)
168
169     # A fresh tracker per chunk ? track IDs only need to be consistent within a
170     # chunk (windows are computed per chunk, then merged across chunks by time).
171     tracker = self.make_tracker(effective_fps)
172     score_thresh = self.config.get("indexing", {}).get("detector_score_threshold",
0.5)
173
174     # Per active frame we keep (frame_idx, peak_velocity, set_of_active_track_ids)
175     # so windows can be enriched with motion stats for logging + fine-tuning.
176     active_frames = []
177     frame_idx = 0
178     track_history: Dict[int, Tuple[float, float, float, float]] = {}
179
180     while True:
181         ret, frame = cap.read()
182         if not ret:
183             break
184
185         # Skip frames for performance ? at 30fps, every 3rd frame still
186         # gives ~10 samples/second, plenty for human-scale velocity.
187         if frame_idx % max(frame_skip, 1) != 0:
188             frame_idx += 1
189             continue
190
191         # Detect persons + assign persistent track IDs.
192         detections = self.detect_and_track(frame, tracker, score_thresh)
193
194         frame_peak_velocity = 0.0
195         frame_active_ids = set()
196         current_ids = set()
197
198         for track_id, bbox in detections:
199             current_ids.add(track_id)
200
201             if track_id in track_history:
202                 prev_bbox = track_history[track_id]
203                 velocity = get_velocity_normalised(prev_bbox, bbox, effective_fps,
frame_width)
204
205                 if velocity > velocity_thresh:
206                     frame_peak_velocity = max(frame_peak_velocity, velocity)
207                     frame_active_ids.add(track_id)
208
209             track_history[track_id] = bbox
210
211         # Prune stale IDs that are no longer visible ? prevents false
212         # velocity spikes when the tracker reassigns an old ID.
213         track_history = {k: v for k, v in track_history.items() if k in current_ids}
214
215         if frame_active_ids:
216             active_frames.append((frame_idx, frame_peak_velocity, frame_active_ids))
217
218         frame_idx += 1
219
220     cap.release()

```

```

221         # Group active frames into continuous windows
222         if not active_frames:
223             return []
224
225         max_gap_frames = int(merge_gap * fps)
226
227         def _finalize(group: List[Tuple[int, float, set]]) -> Dict[str, Any]:
228             velocities = [v for _, v, _ in group]
229             track_ids = set().union(*[ids for _, _, ids in group])
230             return {
231                 "start": group[0][0] / fps + chunk_start,
232                 "end": group[-1][0] / fps + chunk_start,
233                 "active_frame_count": len(group),
234                 "peak_velocity": max(velocities),
235                 "mean_velocity": sum(velocities) / len(velocities),
236                 "track_id_count": len(track_ids),
237                 "chunk_index": chunk_index,
238             }
239
240         windows = []
241         group = [active_frames[0]]
242         for af in active_frames[1:]:
243             if af[0] - group[-1][0] <= max_gap_frames:
244                 group.append(af)
245             else:
246                 windows.append(_finalize(group))
247                 group = [af]
248         windows.append(_finalize(group))
249
250         # Filter out extremely short action windows
251         min_window_duration = self.config.get("indexing",
252 {}).get("min_action_window_duration", 2.0)
253         windows = [w for w in windows if (w["end"] - w["start"]) >= min_window_duration]
254
255         return windows

```

23. assistant (2026-06-13T18:45:30.569Z)

Perfect. Now I have all the information I need. Let me compile the comprehensive mapping:

24. user (2026-06-13T18:46:11.379Z)

Output does not match required schema: root: must have required property 'findings', root: must have required property 'anchors'

25. assistant (2026-06-13T18:46:12.803Z)

Now I'll provide the complete mapping:

26. user (2026-06-13T18:46:46.590Z)

Structured output provided successfully

27. assistant (2026-06-13T18:46:55.770Z)

Summary

I've mapped the **exact live shuttle segmenter architecture** with precise file:line citations:

4-Phase Flow

1. **Phase 1 (Healthcheck)** ? `runner.healthcheck()` at `trajectory\_hybrid.py:141` validates WSL env, TF/PyTorch, GPU

2. **Phase 2 (Trajectory?Windows)** ? ``run_predict()`` produces CSV ? ``parse_trajectory_csv()`` at ``tracknet_runner.py:233`` ? ``trajectory_to_action_windows()`` at ``tracknet_runner.py:252`` seeds windows by grouping high-velocity frames
3. **Phase 3 (AI Handoff)** ? padded clips extracted, threaded through provider, timestamps adjusted, candidate IDs embedded at ``trajectory_hybrid.py:216``
4. **Phase 4 (DB/Link)** ? ``add_play_segment()`` at ``trajectory_hybrid.py:250``, then ``link_candidate_to_segment()`` at ``trajectory_hybrid.py:254``

Window Seeding Method

``trajectory_to_action_windows()`` at ``tracknet_runner.py:252-319`` computes normalized velocity ``(dist/frame_width)*(fps/dframes)``, groups active frames by ``merge_gap`` (default 2.0s), filters by ``min_window_duration`` (default 2.0s).

Persistence & Stats Schema

Call site: ``trajectory_hybrid.py:185-190``
Stats JSON (``trajectory_hybrid.py:179-184``):

```
```json
{"peak_velocity": float, "mean_velocity": float, "active_frame_count": int, "track_id_count": 1}
```
```

Database schema: ``candidate_segments`` table with ``stats`` + ``detection_params`` as JSON columns (``database.py:97-121``)

Subclass Contract

- **family**: config section under ``indexing.*`` (wasb, tracknet)
- **display_label**: human label for logs ("WASB", "TrackNet")
- **build_runner**(``idx_cfg``) at ``trajectory_hybrid.py:53``: returns ``(DetectorRunner, dict_of_extras)``
- **DetectorRunner ABC**: ``run_predict()``, ``healthcheck()``, duck-typed ``parse_trajectory_csv()`` / ``trajectory_to_action_windows()``

FusionSegmenter Seam

Extend ``TrajectoryHybridSegmenter`` and override ``_build_runner()`` to:

- Compose trajectory + person-detector runners ? merge windows in Phase 2 (line 161-165)
- Apply rally-gate filtering (``rally_gate.py:91-112``) post-``trajectory_to_action_windows()``
- Enrich stats dict with fusion metrics before ``add_candidate_segment()`` persistence (line 179-190)
- **Critical**: Preserve ``candidate_ids`` indexing or remap window?candidate linkage for Phase 4 (line 216, 252-254)

Human cue plugs into person-detector producer (``person_detector.py:138-254``); Gate A plugs into rally_gate filter (``rally_gate.py:91-112``).